

Computer Architecture

Brian

September 26, 2026

Contents

0	Foundations	3
0.1	Quantitative Design and Analysis	3
0.1.1	Introduction	3
0.1.2	Classes of Computers	3
0.1.3	Defining Computer Architecture	3
0.1.4	Trends in Technology	4
0.1.5	Trends in Power and Energy in Integrated Circuits	5
0.1.6	Trends in Cost	6
0.1.7	Dependability	6
0.1.8	Measuring, Reporting, and Summarizing Performance	6
0.1.9	Quantitative Principles of Computer Design	6
0.2	Instruction Set Principles	7
0.2.1	Introduction	7
0.2.2	Classifying Instruction Set Architectures	7
0.2.3	Memory Addressing	8
0.2.4	Types and Sizes of Operands	9
0.2.5	Operations in the Instruction Set	9
0.2.6	Instructions for Control Flow	9
0.2.7	Encoding an Instruction Set	11
0.2.8	Cross-Cutting Issues: The Role of Compilers	12
1	Pipelining and Hazards	13
2	Out-of-Order Execution, Branch Prediction, and Speculation	14
3	Store Buffer and Memory Disambiguation	15
4	Performance Measurement: PMUs, TMA, Profiling	16
5	Cache Organization and Locality	17
5.1	Cache Basics	17
5.1.1	Where can a block be placed in a cache?	18
5.1.2	How is a block found if it is in the cache?	19
5.1.3	Which block should be replaced on a cache miss?	19
5.1.4	What happens on a write?	20
5.2	Cache Performance	21
5.2.1	AMAT and processor performance	21
5.2.2	Miss penalty and out-of-order execution processors	21
5.3	Six Basic Cache Optimizations	22

5.3.1	First optimization: larger block size to reduce miss rate	22
5.3.2	Second optimization: larger caches to reduce miss rate	22
5.3.3	Third optimization: higher associativity to reduce miss rate	22
5.3.4	Fourth optimization: multilevel caches to reduce miss penalty	23
6	Virtual Memory, TLBs, and Huge Pages	21
7	Prefetching	22
8	DRAM and NUMA	23
9	Cache Coherence	24
10	Consistency Models, Ordering, and Fences	25
11	Atomics, False Sharing, and Synchronization Design	26
12	Front End and Code Layout	27
13	SIMD and Vectorization	28
14	Low-Latency Platform Tuning	29
15	The I/O Path: PCIe, DMA, Kernel Bypass	30

5 Cache Organization and Locality

Equations

$$2^{\text{index}} = \frac{\text{Cache size}}{\text{Block size} \times \text{Set associativity}}$$

$$\text{CPU execution time} = (\text{CPU clock cycles} + \text{Memory stall cycles}) \times \text{Clock cycle time}$$

$$\text{Memory stall cycles} = \text{Number of misses} \times \text{Miss penalty}$$

$$\text{Memory stall cycles} = \text{IC} \times \frac{\text{Misses}}{\text{Instruction}} \times \text{Miss penalty}$$

$$\frac{\text{Misses}}{\text{Instruction}} = \text{Miss rate} \times \frac{\text{Memory accesses}}{\text{Instruction}}$$

$$\text{Average memory access time} = \text{Hit time} + \text{Miss rate} \times \text{Miss penalty}$$

$$\begin{aligned} \text{CPU execution time} &= \text{IC} \times \left(\text{CPI}_{\text{execution}} + \frac{\text{Memory stall clock cycles}}{\text{Instruction}} \right) \\ &\quad \times \text{Clock cycle time} \end{aligned}$$

$$\begin{aligned} \text{CPU execution time} &= \text{IC} \times \left(\text{CPI}_{\text{execution}} + \frac{\text{Misses}}{\text{Instruction}} \times \text{Miss penalty} \right) \\ &\quad \times \text{Clock cycle time} \end{aligned}$$

$$\begin{aligned} \text{CPU execution time} &= \text{IC} \times \left(\text{CPI}_{\text{execution}} + \text{Miss rate} \times \frac{\text{Memory accesses}}{\text{Instruction}} \right. \\ &\quad \left. \times \text{Miss penalty} \right) \times \text{Clock cycle time} \end{aligned}$$

$$\frac{\text{Memory stall cycles}}{\text{Instruction}} = \frac{\text{Misses}}{\text{Instruction}} \times (\text{Total miss latency} - \text{Overlapped miss latency})$$

$$\begin{aligned} \text{Average memory access time} &= \text{Hit time}_{L1} + \text{Miss rate}_{L1} \times (\text{Hit time}_{L2} \\ &\quad + \text{Miss rate}_{L2} \times \text{Miss penalty}_{L2}) \end{aligned}$$

$$\frac{\text{Memory stall cycles}}{\text{Instruction}} = \frac{\text{Misses}_{L1}}{\text{Instruction}} \times \text{Hit time}_{L2} + \frac{\text{Misses}_{L2}}{\text{Instruction}} \times \text{Miss penalty}_{L2}$$

5.1 Cache Basics

[H&P App. B.1]

- *Cache*: The name given to the first level of memory which is encountered once an address leaves the processor.
 - Caches are also applied whenever buffering is applied to store frequently or recently used items, such as file caches, name caches, etc.
- *Cache hit*: When the processor finds the requested piece of data in the cache; if it is not in a cache it is a *cache miss*.
- *Block*: A fixed size collection of data that is retrieved when accessing a requested word not currently in the cache.
- *Temporal locality*: How likely this word is going to be used in the near future (thus it should be placed in a cache so it can be accessed quickly).
- *Spacial locality*: How likely data near the word accessed is going to be used (thus data in similar locations should be fetched together)
- *Latency*: How long it takes to fetch the first word of a block (time to retrieve)

- *Bandwidth*: How long it takes to fetch the rest of the block (throughput)
- Cache misses are handled by hardware and forces processors to use in-order execution processors to stall, however out-of-order processors may be able to continue with instructions that don't use that piece of data.
- *Page*: A fixed-size block of address space; at a given time each page resides either in main memory or on disk.
- *Page fault*: When a processor references an item not in a cache nor in a page residing in main memory. Handled in software so that the processor does not stall.
- *Memory stall cycles*: The number of cycles the processor is stalled waiting for a memory access.

CPU Execution time = (CPU Clock Cycles + Memory stall cycles) × Clock cycle time

- *Miss penalty*: The number of misses as well as the cost per miss

$$\begin{aligned} \text{Miss Penalty} &= \text{Access latency} + \frac{\text{Block Size}}{\text{Bandwidth}} \\ &\approx \frac{\text{Total Memory Stall Cycles}}{\text{Number of Misses}} \end{aligned}$$

Memory Stall Cycles = Number of misses × Miss penalty

$$\begin{aligned} &= \text{IC} \times \frac{\text{Misses}}{\text{Instruction}} \times \text{Miss penalty} \\ &= \text{IC} \times \frac{\text{Memory accesses}}{\text{Instruction}} \times \text{Miss rate} \times \text{Miss Penalty} \end{aligned}$$

- Note that "Memory Access" includes instruction fetches as well; a program that is 50% memory accesses will have 1.0 accesses for fetching the instruction itself (always happens) + 0.5 accesses for loads/stores = 1.5 memory accesses per instruction.
- The last form is the easiest to measure on hardware. The average miss penalty via the stall cycles and number of misses is empirical and generally used.
- Miss rates can be measured with cache simulators, which take an address trace (a log of all memory addressing operations done) to simulate the cache behaviour and determine the number of total misses and hits.
- Many modern microprocessors have builtin counters that can measure miss rate as well.

5.1.1 Where can a block be placed in a cache?

- If there is a one-to-one mapping between placements and blocks, then the cache is *directly mapped*, with a mapping

$$A \bmod n_B \tag{1}$$

- Where A is the block address and n_B is the number of blocks in the cache.
- If a block can be placed anywhere, then the cache is *fully associative*.
- If a block can be placed in a restricted set of places in the cache, then the cache is *set associative*, where a set is a group of blocks in the cache.
- A block is first mapped to a set (similar to direct mapping) via

$$A \bmod n_S \tag{2}$$

- Where n_S is the number of sets, and the block is placed anywhere in the set.
- The cache placement is then called *n-way set associative*, where n is the number of blocks in a given set.
- Most L1 caches are 8-way and L2/L3 caches often 8- to 16-way.
- Ways and sets are orthogonal to each other

5.1.2 How is a block found if it is in the cache?

- Each block frame has an address tag that gives the block an address.
- Ideally all tags are searched in parallel.
- A *valid bit* is used to say whether or not the entry contains a valid address.
- A full address is divided into 4 sections:
 1. Block Address: Can be further divided into a tag field and index field.
 2. The index field describes which set the block is in
 3. The tag field specifies which block in the set may contain the information.
 4. The block offset selects the desired data from the block.
- The tag field must be searched in parallel since there does not exist a bijective mapping from addresses to block frames (there are too many addresses for not enough blocks).
- While the entire address may be searched, the offset bit would be trivial since the entire block is either present or not (a block cannot be half loaded), and the index is redundant as it was used to select which set the block is in.
- Increasing associativity increases the number of blocks per set and decreases the size of the index which increases the size of the tag. Fully associative caches have no index, and thus have to check every address in parallel.

5.1.3 Which block should be replaced on a cache miss?

When a miss occurs, a block must be selected to be evicted to make space for a different block.

- Random - Candidate blocks are randomly selected to ensure a uniform allocation spread.
- Least recently used (LRU) - Accesses to blocks are recorded, and the block that has been unused for the longest is evicted. This reduces the chances of throwing out information that may be needed soon, based off of the principle that recently used blocks are likely to be used again.
- First in, First out (FIFO) - An alternative to LRU as real LRU is expensive to calculate; FIFO can approximate LRU by determining the oldest block instead of the LRU.

Real processors usually implement a pseudo-LRU; When a way in a given set is accessed, turn on a bit. When all bits in a set are on, turn off all bits except for the most recently accessed item. When a block needs to be evicted, choose randomly from any bit that is off

5.1.4 What happens on a write?

- Since reads are much more common than writes the cache is optimized for reads; processors also need to wait for reads to finish but not necessarily for writes.
- As well, the block addresses are in a separate structure than the actual data itself (split in a tag array and data array). Once the processor indexes into the correct set, the data (at the block offset) in all ways of the set are fetched; there is a n:1 mux at the end which returns the block associated with the correct tag.
- These are done at the same time; if the read is a hit, then the requested part is passed to the processor immediately, and if it is a miss there is no harm to performance (the values read are ignored).
- This cannot be done for writes, as modifying a block cannot be done until the exact block to modify has been found.
- As well, more than enough bytes can be returned on a read; with writes, the exact number of bytes specified are allowed to be modified.
- Two write policies:
 1. *Write through*: Information is written into the block in the cache as well as the block in lower-level memory.
 2. *Write back*: Information is only written into the block in the cache; the block is then only written to the next level down when evicted.
- For write-back caches, they also often have a *dirty bit*, which signifies whether that block has been modified; if it is clean, then there is no need to write the block back into the next level down (since it has not been changed).
- Write-back is better for latency as multiple writes within a block require only one write to the lower level memory.
- Write through is simpler to implement as the cache block is always clean; write through also guarantees that the next level down will always contain the most recent copy of the data (easier for coherency).
- *Write stall*: When the processor must wait for writes to complete.
- *Write buffer*: Write instructions are queued in a buffer, overlapping processor execution with memory updating (write stalls are still possible!)
- Since the processor doesn't need the data after a write, there are two options on a write miss:
 1. *Write allocate*: On write miss, allocate the block and proceed with the write hit actions. Write misses act exactly like read misses.
 2. *No-write allocate*: On write miss, modify the block in-place in the lower-level memory

Assuming fully associative write-back that starts empty:

```
1 Write Mem[100]; // 1
2 Write Mem[100]; // 2
3 Read  Mem[200]; // 3
4 Write Mem[200]; // 4
5 Write Mem[100]; // 5
```

For no-write allocate:

1. Miss
2. Miss (Mem[100] was not allocated)
3. Miss
4. Hit
5. Miss

For write-allocate:

1. Miss
2. Hit (Mem[100] was allocated)
3. Miss
4. Hit
5. Hit

5.2 Cache Performance

Memory hierarchy performance is often measured via *Average memory access time*:

$$\text{Average memory access time} = \text{Hit time} + \text{Miss rate} \times \text{Miss penalty} \quad (3)$$

- Where hit time is the time to hit a cache.

5.2.1 AMAT and processor performance

- For in-order execution processors, AMAT is an accurate representation of processor performance as it correlates strongly with memory stall cycles.
- Recall CPU time formula:

$$\text{CPU time} = (\text{CPU execution clock cycles} + \text{Memory stall cycles}) \times \text{Clock cycle time} \quad (4)$$

- Hit clock cycles (number of cycles for a cache hit) is widely accepted to be included in CPU execution clock cycles.

5.2.2 Miss penalty and out-of-order execution processors

- As OoO processors can overlap instructions to reduce the apparent latency, miss penalty becomes more ambiguous to define.
- Redefine memory stall cycles per instruction:

$$\frac{\text{Memory stall cycles}}{\text{Instruction}} = \frac{\text{Misses}}{\text{Instruction}} \times (\text{Total miss latency} - \text{Overlapped miss latency}) \quad (5)$$

- To consider only the exposed cycles where the processor must stall.
- The length of memory latency (what is considered as the beginning and end of a memory operation) and the length of latency overlap are not strictly defined due to the complexity of OoO processors.
- We can define the stalled clock cycles as the number of cycles in which the processor could not retire the maximum number of instructions in that cycle (This includes overlapped instructions).

- We can measure latency starting from when the memory instruction is queued to the moment it is actually seen by the memory system.

5.3 Six Basic Cache Optimizations

As AMAT is a function of hit time, miss rate, and miss penalty, there are 3 categories of potential optimizations:

1. Reduce the miss rate: larger block/cache size, higher associativity (increases number of ways per allowed set)
2. Reduce the miss penalty: Multilevel caches and read priority over writes
3. Reduce the time to hit in the cache: index cache with virtual address

Misses can be sorted into three categories:

1. Compulsory: The very first access to a block can never be in the cache and must be brought in. also called *cold-start* or *first-reference misses*

Misses can be sorted into three categories:

1. *Compulsory*: The very first access to a block can never be in the cache and must be brought in. also called *cold-start* or *first-reference misses*.
2. *Capacity*: If the cache cannot fit all blocks needed during execution, capacity misses will occur when blocks get evicted and later retrieved.
3. *Conflict*: In set associative or direct map, misses will occur if too many blocks map to the same set. These are also known as *collision misses*.

(Coherency misses due to cache flushes are not covered yet.)

5.3.1 First optimization: larger block size to reduce miss rate

- Larger blocks reduce compulsory misses (less first accesses as blocks are larger) and can exploit spatial locality better.
- Larger block however will increase conflict misses and even capacity misses (given a small cache size).
- Block size depends on both latency and badwidth;
 - High latency and high bandwidth encourages large block sizes as the cache receives more bytes per miss for a small increase in miss penalty
 - Low latency and low bandwidth encourages small block sizes as there is less time saved from a larger block.

5.3.2 Second optimization: larger caches to reduce miss rate

- Reduces capacity misses, with the drawback of longer hit times and higher cost and power.

5.3.3 Third optimization: higher associativity to reduce miss rate

- Increasing associativity will decrease the number of conflict misses.
- In general, a direct-mapped cache of size N has approx. the same miss rate as a two-way set associative cache of size $N/2$.

5.3.4 Fourth optimization: multilevel caches to reduce miss penalty

- Adding a level of cache allows the first-level cache to be small enough to match the clock speed of the processor, and the second-level cache can be large enough to capture accesses that would otherwise go to main memory, improving the miss penalty.
- AMAT for a two-level cache:

$$\text{AMAT} = \text{Hit time}_{L1} + \text{Miss rate}_{L1} \times (\text{Hit time}_{L2} + \text{Miss rate}_{L2} \times \text{Miss penalty}_{L2}) \quad (6)$$

- *Local miss rate*: Number of misses in a given cache divided by total memory accesses to that cache. For first-level cache it is Miss rate_{L1} , and for second-level it is Miss rate_{L2} .
- *Global miss rate*: Number of misses in a given cache divided by total memory accesses generated by the processor. For the first-level cache it is still Miss rate_{L1} and for the second-level cache it is $\text{Miss rate}_{L1} \times \text{Miss rate}_{L2}$.

B-31